Assertions

Checking your own assumptions while you build

Exceptions handle problems a program expects to meet: bad files, bad input, and careless callers. This part covers a different tool. An *assertion* checks an assumption that should never be false if your own code is correct. Assertions are small and easy to misuse, so these notes cover what they do, why they are worth using, and when an exception is the better choice. For reading, see Core Java, Volume I, Chapter 7.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. What an Assertion Is

An *assertion* states something you believe must be true at a certain point in your code. If it is false, your code has a bug. Java has two forms:

```
assert condition;
assert condition : message;
```

If the condition is false, Java throws an `AssertionError`, using the message if you gave one. Here, a private helper assumes it is never handed an empty array:

```
private static double average(double[] values) {
    assert values.length > 0 : "average of an empty array";
    double total = 0;
    for (double v : values) {
        total += v;
    }
    return total / values.length;
}
...
System.out.println(average(new double[] {5000, 7500}));
System.out.println(average(new double[0]));
```

2. Assertions Are Off by Default

Here is the surprise: run that program normally, and the assertion does nothing.

```
6250.0
NaN
```

Assertions are disabled unless you turn them on with the `-ea` option (short for `-enableassertions`). With it, the bug is caught at the line that assumed too much:

```
$ java -ea AssertDemo
6250.0
Exception in thread "main" java.lang.AssertionError: average of an empty array
        at AssertDemo.average(AssertDemo.java:3)
        at AssertDemo.main(AssertDemo.java:13)
```

In Eclipse, open **Run**, then **Run Configurations**, select your program, and add `-ea` under **VM arguments** on the **Arguments** tab. The design is deliberate: assertions are a testing and debugging tool. Developers turn them on while they work, and the program runs at full speed in production with them off.

3. Why Bother with Assertions?

Students often ask what the point is, if assertions do nothing unless someone turns them on. There are four good answers.

- **They catch a bug where it starts, not where it shows up.** With assertions off, the `average` example quietly returns `NaN`. That value can flow into a total and then into a report, and the strange number appears several methods later with no clue where it came from. With assertions on, the program stops at the exact line whose assumption was wrong, and the message says which assumption it was.
- **They are documentation that checks itself.** A comment such as "values is never empty here" can become false after an edit, and nothing tells you. An assertion states the same assumption in a form Java checks every time you test, and it tells the next reader what the method counts on.
- **They cost nothing in the finished program.** Because they are off by default, you can check things you would not want to pay for in every run, such as confirming that a list is still sorted after each insert. A carpenter checks a wall with a level while building it. The finished house does not carry the level around, but the wall is straight because someone checked.
- **They have a different job from exceptions.** Exceptions protect a program from the outside world: bad files, bad input, and careless callers. Assertions protect you from your own mistakes, the things in your own logic that "cannot happen."

There is an honest other side. Many professional teams rarely use the `assert` statement, because it is easy to forget `-ea`, and then the checks never run. Many prefer checks that always run, such as `Objects.requireNonNull` or throwing an `IllegalStateException`, along with unit tests. You will also meet methods such as `assertEquals` in JUnit, a testing library. Those are test methods, not the `assert` statement, despite the similar name.

The takeaway: the `assert` keyword is a small debugging tool, but the habit behind it matters a great deal. Notice what your code assumes, write the assumption down, and check it.

4. When to Use an Assertion

Because assertions may be turned off, they can only check things that would be bugs in your own code. They are a poor fit for anything that could legitimately happen while the program runs.

Table 1: Choosing between an assertion and an exception.

Situation	Use
A public method receives a bad argument	An exception, such as `IllegalArgumentException`. Callers you do not control can pass anything, and the check must run every time.
A file is missing or a line in it is bad	An exception. This is a problem the program expects to meet.
A private helper assumes something about the values its own class passes it	An assertion is fine. Only your code calls it.
A branch of code that "cannot happen," such as the `default` of a `switch` that covers every case	An assertion, or an exception if you want the check to run in production too.

Never put work inside an assertion. When assertions are off, the whole statement is skipped, including any method call inside the condition. If you write `assert staff.remove(ada);`, Ada is removed while you test with `-ea`, and stays in the list when the program runs without it. Do the work first, store the result, and assert about the result.

An `AssertionError` is an `Error`, not an `Exception`. Do not catch it. It means the program's own logic is broken, and the fix is in the code.

5. Summary

Table 2: Assertions in brief.

Topic	Key points
What it is	`assert condition;` or `assert condition : message;` throws an `AssertionError` if the condition is false.
When it runs	Only when the program runs with `-ea`. Otherwise the whole statement is skipped.
Why use it	Catches a bug where it starts, documents an assumption in a form that is checked, and costs nothing in the finished program.
When not to use it	Never to check arguments to public methods, never for problems the program expects to meet, and never with side effects.

6. Try It

1. Add an assertion to a private method in one of your own classes. Run the program with and without `-ea`, and pass it a value that breaks the assertion. What changes?
2. Write a short program that runs `assert staff.remove(ada);` on a list of names. Print the list after the assertion, and run the program with and without `-ea`. Why do the two runs print different lists?